

Звіт

до лабораторної роботи №6
Об'єктно-орієнтоване програмування. Java
Поліморфізм

Студент: Ісаєв Захар Ігорович

Група: КН-924В

Викладач: Володимир Миколайович Савченко

Харків - 2026

Мета роботи

Сформуванати практичні навички проєктування семантично коректних поліморфних моделей у Java, використання абстрактних класів як узагальнених типів, правильного перевизначення методів і виклику поведінки через посилання базового типу без використання інтерфейсів.

Опис предметної галузі

У лабораторній роботі розглядається предметна галузь освітнього оцінювання. Вона пов'язана з перевіркою результатів навчання студента за допомогою різних форм контролю: іспиту, тестового оцінювання та курсового проєкту.

Основною сутністю моделі є Assessment, тобто загальне оцінювання. Для всіх видів оцінювання спільними є назва курсу, ім'я студента, група, максимальний бал і набраний результат. Однак кожен тип оцінювання має власні правила перевірки успішності.

У роботі клас Assessment використовується як абстрактний базовий клас, а Exam, QuizAssessment і CourseProject є його конкретними підкласами. Поліморфізм проявляється в тому, що всі ці об'єкти можна обробляти через тип Assessment, але метод evaluateResult() для кожного з них працює по-різному.

Побудова поліморфічної моделі

Абстрактний базовий клас Assessment

Клас Assessment є узагальненим типом для всіх видів освітнього оцінювання. У ньому зберігаються спільні поля: assessmentId, courseName, studentName, studentGroup, maxScore і earnedScore. Також у цьому класі реалізовано спільні методи getPercentage(), hasMinimumPercentage() і summary().

Метод evaluateResult() оголошено як абстрактний. Саме він визначає спільну поліморфну операцію, яку кожен підклас реалізує по-своєму.

Клас Exam

Клас Exam описує іспит і успадковує Assessment. Він має додаткові поля durationMinutes і proctored, які зберігають тривалість іспиту та інформацію про прокторинг.

У цьому класі перевизначено метод evaluateResult(). Для успішного складання іспиту потрібно мати прокторинг і набрати не менше 60%.

Клас QuizAssessment

Клас QuizAssessment описує тестове оцінювання. Він має поля attemptNumber, maxAttempts і timeLimitMinutes, які відповідають за кількість спроб і часовий ліміт.

Метод evaluateResult() перевизначено так, щоб враховувати ліміт спроб і мінімальний результат 50%. Якщо спроб забагато або бал нижчий за мінімум, тест не зараховується.

Клас CourseProject

Клас CourseProject описує курсовий проєкт. Він містить поля defensePassed і repositoryLink, які показують, чи пройдено захист і де зберігається робота.

У методі evaluateResult() перевіряється захист і мінімальний результат 70%. Якщо захист не пройдено або бал недостатній, курсовий проєкт не зараховується.

Клас Main

Клас Main демонструє роботу поліморфної моделі. У ньому об'єкти Exam, QuizAssessment і CourseProject зберігаються в масиві типу Assessment[].

У циклі для кожного об'єкта викликається метод evaluateResult(). Виклик виконується через базовий тип Assessment, але результат залежить від реального класу об'єкта. Це і є проявом поліморфізму в програмі.

Механізми Java та їх приклади використання

abstract - використання класу Assessment як абстрактного узагальненого типу для всіх видів освітнього оцінювання

`extends` - успадкування класів `Exam`, `QuizAssessment` і `CourseProject` від абстрактного класу `Assessment`

`super` - виклик конструктора базового класу `Assessment` у конструкторах підкласів для ініціалізації спільних полів

`@Override` - перевизначення методу `evaluateResult()` у класах `Exam`, `QuizAssessment` і `CourseProject`

поліморфізм - робота з об'єктами різних підкласів через масив базового типу `Assessment[]` у класі `Main`

`final` - використання `final` для полів класів, наприклад `assessmentId`, `courseName`, `studentName`, `studentGroup`, `maxScore`, `earnedScore`, щоб після створення об'єкта ці значення не змінювались

Аналіз моделі з ЛР5

Для аналізу використовую лаб. №5, у якій реалізовано ієрархію медичних працівників: `MedicalWorker`, `Doctor`, `Surgeon`, `Pediatrician` і `CardiacSurgeon`.

У цій моделі можна коректно ввести поліморфну абстракцію, оскільки всі ці класи описують різні види медичних працівників. Узагальненим типом може бути абстрактний клас `MedicalWorker`, а конкретними або спеціалізованими формами цього типу можуть бути `Doctor`, `Surgeon`, `Pediatrician` і `CardiacSurgeon`.

Спільний стан, який можна винести до базового класу `MedicalWorker`:

`fullName` — повне ім'я медичного працівника;

`hospitalName` — назва лікарні;

`experienceYears` — кількість років досвіду.

Спільною поведінкою може бути метод `description()`, який формує текстовий опис об'єкта. У базовому класі `MedicalWorker` цей метод можна зробити абстрактним або залишити загальну частину опису в окремому методі, наприклад `baseDescription()`, а метод `description()` перевизначати в кожному підкласі.

У такій моделі поліморфізм проявляється в тому, що об'єкти Doctor, Surgeon, Pediatrician і CardiacSurgeon можуть зберігатися у масиві або списку типу MedicalWorker, але під час виклику методу description() буде виконуватися реалізація відповідного реального класу.

Приклад можливої поліморфної роботи:

```
MedicalWorker[] workers = {  
    new Doctor(...),  
    new Surgeon(...),  
    new Pediatrician(...),  
    new CardiacSurgeon(...)  
};  
  
for (MedicalWorker worker : workers) {  
    System.out.println(worker.description());  
}
```

У цьому прикладі змінна worker має тип MedicalWorker, але метод description() викликається відповідно до фактичного типу об'єкта. Це означає, що така модель підтримує поліморфізм.

Висновок

Модель з лабораторної роботи №5 можна коректно розширити до поліморфної абстракції, оскільки всі розглянуті класи є спеціалізованими формами медичного працівника. Спільний стан доцільно залишити в базовому класі MedicalWorker, а метод description() використовувати як спільну поліморфну операцію. Кожен підклас може перевизначати цей метод і додавати до опису власні специфічні дані. Такий підхід дозволяє працювати з різними медичними працівниками через один базовий тип MedicalWorker.

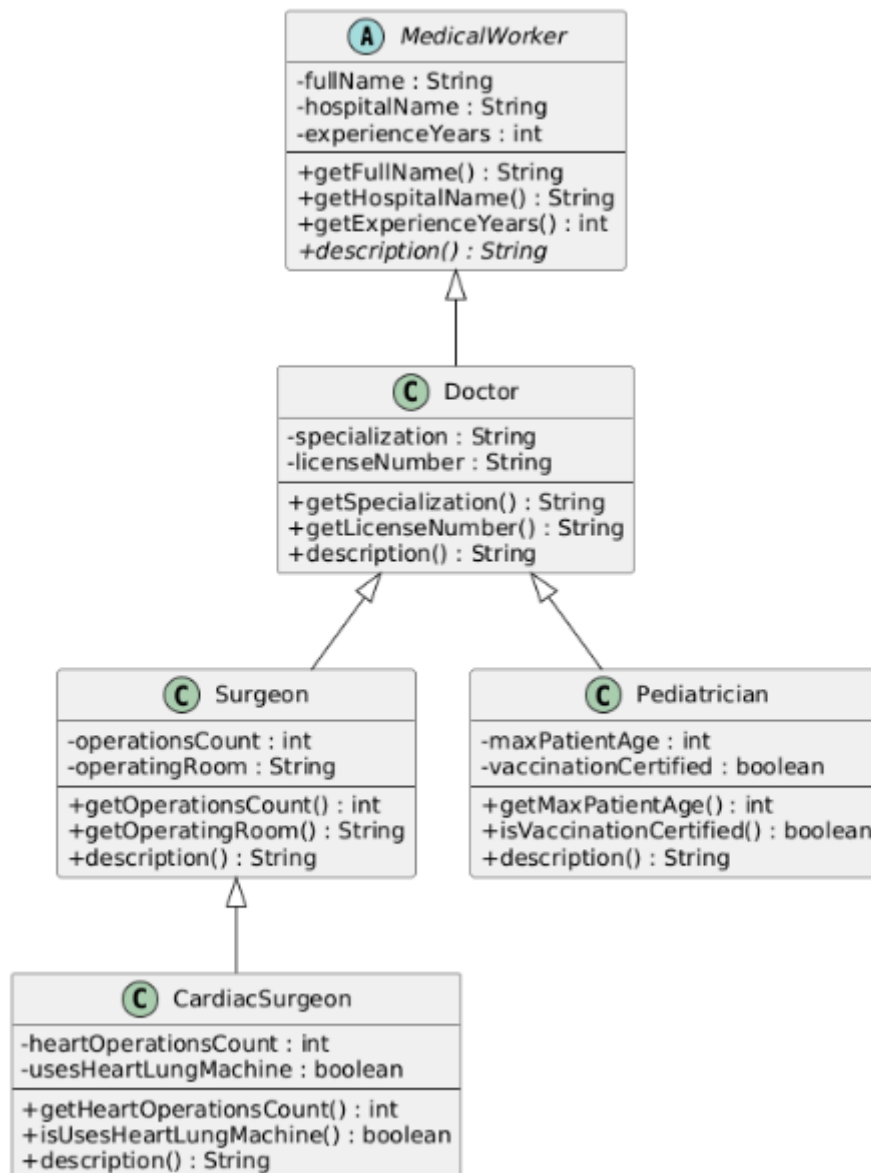


Рис. 1 - PlantUML до 5 роботи

Демонстрація роботи лаб №6

Під час виконання програми створюються об'єкти різних видів освітнього оцінювання: іспит, тестове оцінювання та курсовий проєкт. Усі ці об'єкти зберігаються в масиві базового типу `Assessment`, після чого програма виводить коротку інформацію про кожне оцінювання та результат його перевірки. Для кожного підкласу метод `evaluateResult()` працює по-різному: іспит враховує прокторинг і мінімальний бал, тестове оцінювання — кількість спроб, а курсовий проєкт — проходження захисту. Ця демонстрація підтверджує, що програма працює з різними видами оцінювання через спільний тип `Assessment`, тобто поліморфно.

```

EX-101 | Object-Oriented Programming | Zakhar Isaiev | KN-924V | 78.0/100.0 points | 78.0%
Exam passed: final exam score is acceptable

QZ-205 | Object-Oriented Programming | Zakhar Isaiev | KN-924V | 13.0/20.0 points | 65.0%
Quiz passed: minimum quiz score is reached

CP-310 | Object-Oriented Programming | Zakhar Isaiev | KN-924V | 82.0/100.0 points | 82.0%
Course project passed: implementation and defense are accepted

```

Рис. 2 - Приклад роботи програми

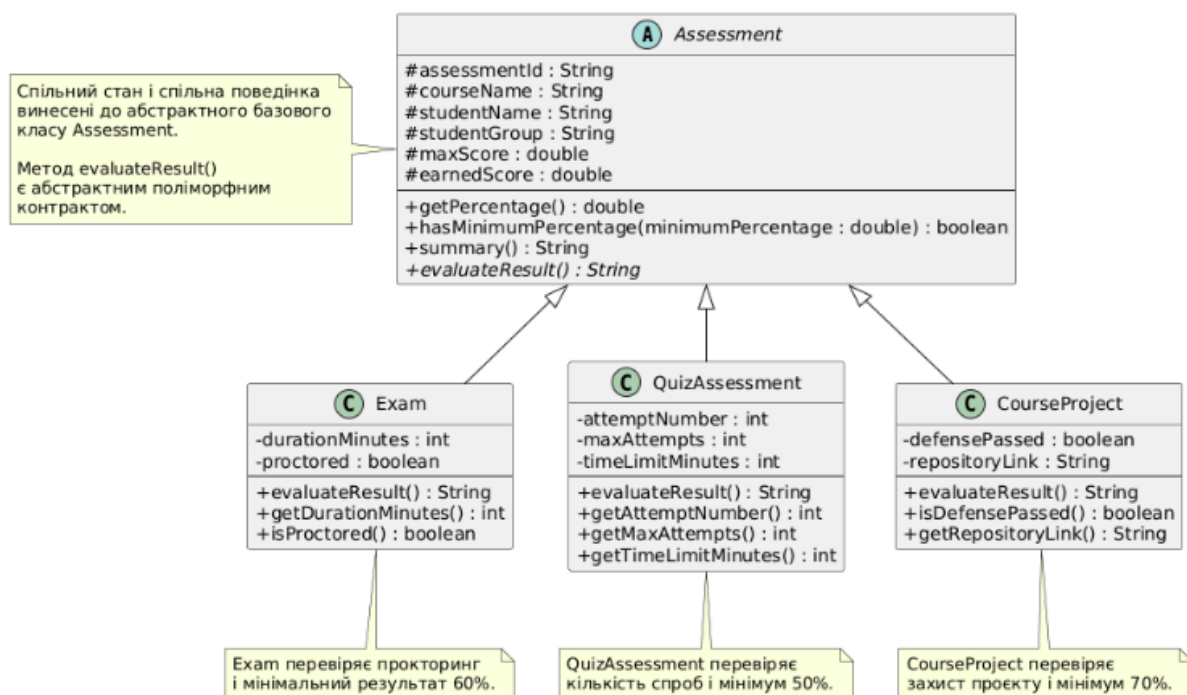


Рис. 3 - PlantUML до 6 роботи

У лабораторній роботі №6 було реалізовано поліморфну модель освітнього оцінювання з використанням абстрактного класу Assessment. Було створено три конкретні підкласи: Exam, QuizAssessment і CourseProject, які мають спільну основу, але по-різному визначають результат оцінювання. У програмі продемонстровано поліморфний виклик методу evaluateResult() через масив базового типу Assessment[]. Виконана робота показує, як за допомогою абстрактних класів і перевизначення методів можна обробляти різні об'єкти однаковою способом, зберігаючи їхню власну поведінку.